

Article

Intelligent Service Chain Orchestration and Resource Allocation in End-Edge Collaborative IIoT Using Multi-Agent Proximal Policy Optimization

Tianzhen Zhao ^{1,†}, Bingxin Tian ^{2,†}, Lei Wang ², Wanming Ma ² and Bin Wei ^{2,*}¹ School of Electronic Engineering, Xidian University, Xi'an 710126, China; 23009100123@stu.xidian.edu.cn² China Mobile Research Institute, Beijing 100053, China; tianbingxin@chinamobile.com (B.T.)

* Correspondence: weibin@chinamobile.com

† These authors contributed equally to this work.

Abstract

The massive heterogeneous data streams and stringent low-latency requirements in the Industrial Internet of Things (IIoT) pose new challenges for edge network resource management. This paper addresses the joint optimization problem of Service Function Chain (SFC) orchestration and resource allocation in edge gateway-assisted IIoT networks, formulated as a mixed-integer nonlinear programming (MINLP) model to minimize end-to-end latency and energy consumption while satisfying quality-of-service (QoS) constraints. To tackle this NP-hard problem and the challenges of partial observability in distributed environments, we propose the SFC Orchestration and Resource Allocation-based Multi-Agent Proximal Policy Optimization (SORA-MAPPO) algorithm. The algorithm adopts a centralized training with decentralized execution (CTDE) paradigm with an intelligent agent cooperation mechanism. Simulation results validate the effectiveness of the proposed scheme in complex IIoT scenarios.

Keywords: Industrial Internet of Things (IIoT); Service Function Chain (SFC) orchestration; multi-agent deep reinforcement learning (MADRL); edge computing; resource allocation

1. Introduction

With the advent of fifth-generation (5G) mobile communication technology, the Industrial Internet of Things (IIoT) is driving a profound transformation of manufacturing towards Industry 4.0 [1], whose diverse applications generate massive heterogeneous data streams with stringent low-latency processing requirements [2]. To address the inherent high transmission latency of remote cloud computing [3], mobile edge computing (MEC) offers an effective solution by deploying computing resources at the network edge to meet the demanding latency needs of IIoT applications. However, traditional network architectures still rely on expensive dedicated hardware [4], making it challenging to flexibly adapt to diverse service demands and resulting in high operational costs. Network function virtualization (NFV) decouples network functions from hardware [5], allowing services to be constructed as flexibly orchestratable Service Function Chains (SFCs) composed of virtual network functions (VNFs), significantly enhancing network flexibility and scalability. This SFC-based approach demonstrates broad applicability across various domains: in urban intelligent transportation systems, vehicle-to-infrastructure communications require orchestrated VNF chains for real-time traffic optimization, collision avoidance, and route planning [6]; in smart campus networks, integrated Internet of Things (IoT) sensors and



Academic Editors: Guandong Yu and Shengli Liu

Received: 22 April 2026

Revised: 23 May 2026

Accepted: 1 June 2026

Published: 4 June 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

edge gateways coordinate through SFC orchestration to provide seamless connectivity for educational applications, environmental monitoring, and security management [7]; in industrial manufacturing, production line sensors and robotic systems leverage SFC-enabled edge computing for predictive maintenance, quality control, and adaptive manufacturing processes [8]. In such distributed collaborative networks composed of multiple edge gateways that simultaneously host both MEC platforms and VNFs, jointly optimizing VNF orchestration and resource allocation while satisfying quality-of-service (QoS) constraints has become a critical research challenge.

In IIoT environments, computational task offloading to edge gateways is considered a key technology for addressing the limited computing capabilities of resource-constrained terminal devices. Existing research works primarily focus on the joint optimization of task allocation and resource scheduling strategies. Representative studies utilize Lyapunov optimization techniques for delay-aware energy-efficient offloading algorithms [9], design blockchain-enabled computation offloading and resource pricing schemes via Stackelberg game theory [10], and construct comprehensive stochastic computation offloading frameworks and MEC architectures that consider task priority constraints [11,12].

Furthermore, to adapt to highly dynamic network states, recent cutting-edge studies increasingly adopt advanced deep reinforcement learning (DRL) algorithms for intelligent resource management. Specifically, these works propose DRL-based intelligent offloading for blockchain-enabled systems [13], develop personalized federated DRL (PFDRL) for joint offloading and resource allocation in multi-edge environments [14], and design scalable offloading methods combining Proximal Policy Optimization (PPO) with Differentiable Neural Computers (DNCs) to handle complex computation tasks [15]. However, most of these studies overlook the specific requirements and processing order of individual sub-functions within complex services, making their approaches unsuitable for scenarios requiring sophisticated SFC orchestration [16].

SFC orchestration aims to efficiently embed ordered VNF sequences into NFV-enabled IIoT infrastructure while optimizing system performance [17]. Recent works propose online deployment algorithms that optimize latency by prioritizing adjacent paths and considering dynamic network load conditions [18], investigate SFC deployment problems under network function parallelization scenarios by flexibly adjusting parallel VNF resource allocation strategies [19], and study the optimization of SFC deployment and dynamic resource allocation based on VNF performance-resource functions in cloud-edge collaborative environments [20]. However, SFC orchestration is inherently an NP-hard combinatorial optimization problem [21], and traditional optimization methods relied upon by the aforementioned studies face significant computational bottlenecks. Methods based on integer linear programming (ILP) suffer from prohibitively high computational complexity, making it difficult for them to meet real-time requirements in dynamic environments, while heuristic algorithms are prone to getting trapped in local optima with no guarantees of solution quality [22].

Considering the inherent stochasticity of service requests and the dynamic nature of network states in complex IIoT scenarios, DRL has demonstrated great potential for handling such high-dimensional resource management problems due to its online learning capabilities and adaptive decision-making mechanisms [23,24]. Recent works utilize deep Q-networks (DQNs) to achieve distributed SFC embedding in edge computing environments [25], employ natural Actor-Critic algorithms to minimize service latency through joint optimization of SFC routing and wireless resource orchestration among IIoT servers [26], and adopt hierarchical hybrid continuous and discrete action (HHFDA) DRL methods to achieve joint optimization of computational resources and VNF deployment strategies [27]. However, these existing studies often decompose the complex

end-to-end SFC deployment problem into a series of independent single-step decisions, thereby ignoring the deep collaborative dilemma caused by partial observability and the intricate sequential dependencies among multi-node VNF deployment decisions in truly distributed environments.

To address the aforementioned challenges and limitations, this paper constructs a unified joint-optimization model in a multi-edge-gateway-assisted IIoT system, where static edge gateways simultaneously serve as both MEC platforms and NFV nodes, aiming to minimize the weighted total cost composed of end-to-end latency and system energy consumption through cooperative optimization of VNF deployment and computational resource allocation. We propose a novel Multi-Agent Proximal Policy Optimization (MAPPO) algorithm based on a multi-agent deep reinforcement learning (MADRL) framework under the centralized training with decentralized execution (CTDE) paradigm, enabling individual agents to effectively utilize global state information to assist learning during the training phase, while requiring only their local observations to make fast and efficient distributed collaborative decisions regarding VNF deployment and computation offloading during the execution phase, thereby achieving optimal responses to dynamic service demands under fixed network topologies.

The main contributions of this article are as follows:

- IIoT services are heterogeneous, spanning ultra-low latency to compute-intensive tasks, which complicates resource offloading and allocation. In an edge gateway-IIoT device cooperative network, SFCs are virtualized into VNFs and offloaded to nodes for flexible orchestration, while scheduling must also address the combined challenges of energy use, end-to-end delay and QoS requirements. A multi-node model is thus established to minimize total system cost by jointly addressing VNF deployment decisions and computation resource allocation, achieving integrated optimization of computation, and deployment.
- To address the challenges of dynamic service demands and large-scale node collaboration in IIoT, this paper proposes the SFC Orchestration and Resource Allocation-based Multi-Agent Proximal Policy Optimization (SORA-MAPPO) algorithm. By integrating SFC orchestration and resource allocation decisions into a unified reinforcement learning framework, SORA-MAPPO uniformly models edge gateways and IIoT devices as independent, cooperative agents and adheres to the CTDE paradigm to learn the optimal policy for complex service processing.
- A multi-edge-gateway-assisted IIoT simulation platform is constructed, and through multi-scenario comparative experiments and hyperparameter tuning, the effectiveness and robustness of SORA-MAPPO in complex environments are validated from multiple dimensions, providing a comprehensive performance evaluation benchmark for SFC orchestration and resource allocation in IIoT scenarios.

2. System Model and Problem Formulation

2.1. Network Model

As shown in Figure 1, this paper constructs a multi-level distributed computing network for IIoT. This network is mainly composed of the underlying IIoT layer and the upper edge gateway layer. Among these, the IIoT layer includes various terminal devices such as sensors, cameras, robots, and robotic arms, which are responsible for generating diverse business data and computing tasks. Above this, a set of edge gateways constitutes a fixed, distributed edge computing platform, which collaborates with terminal devices through wireless links to complete flexible computing offloading and resource allocation. In order to efficiently meet the differentiated requirements between delay-sensitive services (such as robot patrol) and resource-intensive services (such as intelligent monitoring), this

paper abstracts the complex business process into SFCs. As shown in the figure, each SFC consists of a series of ordered VNFs.

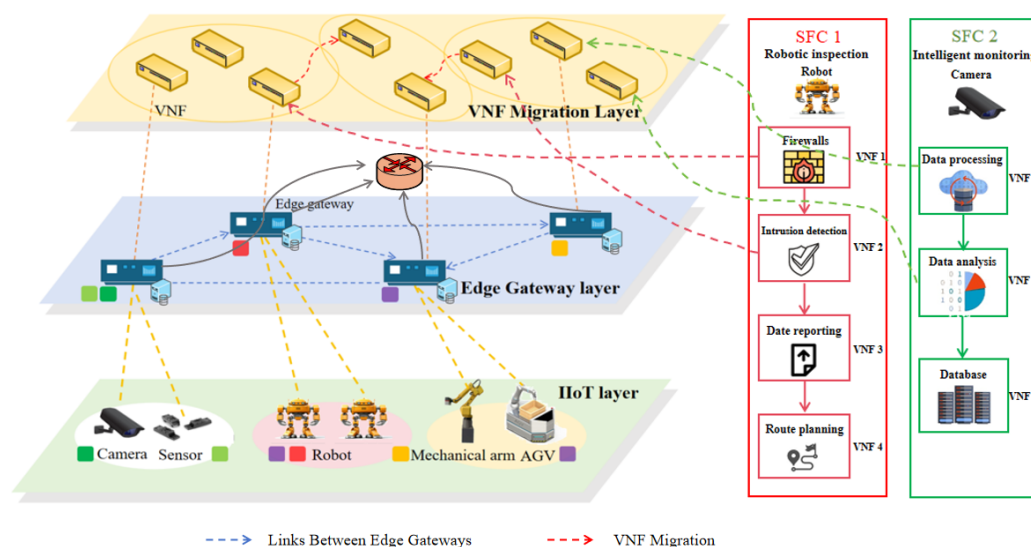


Figure 1. SFC orchestration strategy in multi-edge-gateway-assisted IIoT networks.

By virtualizing the business functions in the SFC into independent VNF modules, their deployment is no longer restricted by the underlying hardware. This enables each VNF to be used as an independent computing task and be flexibly deployed, instantiated, or migrated to any available node in the network, thus greatly improving the resource utilization efficiency and business processing flexibility of the system. To intuitively illustrate the dynamic interaction process of the proposed network model during SFC request processing, Figure 2 depicts its end-to-end service flow. The process demonstrates how an SFC request is initiated by an IIoT device, collaboratively processed by multiple intelligent nodes through VNF orchestration, deployment, computing allocation, and task offloading, and finally returns the processing results to the requesting IIoT device.

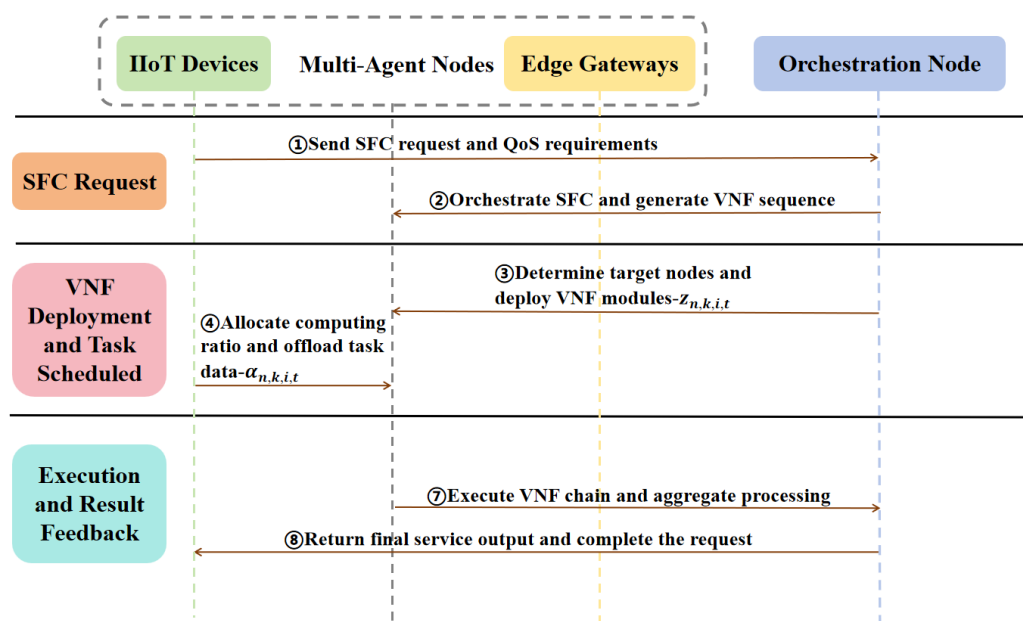


Figure 2. Collaborative multi-agent task flow.

This paper models this hybrid network composed of edge gateways and IIoT devices as a time-varying graph $P = (\mathcal{V}, \mathcal{E})$. Here, the vertex set $\mathcal{V}$ represents all available computing nodes, which are jointly composed of the IIoT terminal device set $\mathcal{D}$ and the edge gateway node set $\mathcal{G}$; that is, $\mathcal{V} = \mathcal{D} \cup \mathcal{G}$. Any computing node $i \in \mathcal{V}$ in the network, whether it is an IIoT device $d \in \mathcal{D}$ or an edge gateway $g \in \mathcal{G}$, is defined by its inherent computing capacity C_i , CPU processing rate F_i , and maximum storage capacity S_i . The edge set $\mathcal{E}$ represents all available wireless communication links between these nodes, and its bandwidth capacity changes according to the relative positions and channel conditions between nodes.

2.2. SFC Model

In a dynamic and resource-constrained edge gateway-assisted IIoT environment, the core of service guarantee lies in the efficient orchestration and management of a series of heterogeneous SFC requests. This paper models each SFC request $\mathcal{R}_n$ as a tuple $\mathcal{R}_n = \{\mathcal{F}_n, \mathcal{C}_n, \mathcal{S}_n, \mathcal{L}_n, \Delta_n, \text{type}_n, \dots\}$, the core of which is an ordered sequence composed of multiple VNFs, denoted as $\mathcal{F}_n = \{v_{n,1}, v_{n,2}, \dots, v_{n,|\mathcal{F}_n|}\}$. In addition, $\mathcal{C}_n$ represents the inherent computing complexity of each VNF, that is, the number of CPU cycles required to process each bit of data, denoted as $\mathcal{C}_n = \{c_{n,1}, c_{n,2}, \dots, c_{n,|\mathcal{F}_n|}\}$. Similarly, $\mathcal{S}_n$ represents the storage resource requirement of each VNF, denoted as $\mathcal{S}_n = \{s_{n,1}, s_{n,2}, \dots, s_{n,|\mathcal{F}_n|}\}$, where $s_{n,k}$ encompasses both the static VNF image footprint and the dynamic state data generated during task execution. $\mathcal{L}_n = \{l_{n,1}, l_{n,2}, \dots, l_{n,|\mathcal{F}_n|}\}$ represents the data packet lengths that each SFC needs to process from IIoT devices, and the length of data packets may change after being processed by a VNF. $l_{n,0}$ and $l_{n,j}$ represent the initial data packet length and the data packet length after being processed by VNF $v_{n,j}$ respectively.

All these complex processing and transmission activities must be completed within the maximum end-to-end delay Δ_n tolerable by the SFC request. To ensure that the constraint of QoS is satisfied, define $T_{n,t}$ as the actual end-to-end delay of the SFC request $\mathcal{R}_n$ in time slot t , which consists of processing delay, transmission delay, VNF instantiation delay, etc. Therefore, this QoS constraint can be expressed as $T_{n,t} \leq \Delta_n$. To manage different types of services in a refined manner, the type_n attribute divides SFC requests into two categories: one is applications that are extremely sensitive to delay (such as remote real-time control of drones), marked as $\text{type}_n = 1$; the other is computing or data-intensive tasks (such as high-definition video monitoring and analysis), marked as $\text{type}_n = 0$.

In order to optimize the deployment process of the SFC, it is necessary to model it as a set of decision variables. This paper defines the VNF deployment decision variable $z_{n,k,i,t} \in \{0, 1\}$. When $z_{n,k,i,t} = 1$, it means that the k -th VNF of the SFC request $\mathcal{R}_n$ is specifically deployed on the network node i at the time slot t ($i \in \mathcal{V}$). However, relying solely on binary decision variables $z_{n,k,i,t}$ can only determine the deployment of different VNFs in the SFC on different nodes, but it is difficult to accurately describe how multiple nodes jointly share the computing load of the same VNF. Therefore, this paper introduces a computing scheduling ratio variable $\alpha_{n,k,i,t} \in [0, 1]$, which represents the proportion of all computing tasks of VNF $v_{n,k}$ borne by node i . The following constraints are satisfied:

$$\sum_{i \in \mathcal{V}} \alpha_{n,k,i,t} = 1 \quad \forall n, k, \quad (1)$$

$$\alpha_{n,k,i,t} \leq z_{n,k,i,t} \quad \forall n, k, i, t. \quad (2)$$

For ease of reference, all notations and variables introduced in the network model and SFC model are consolidated in Table 1.

Table 1. Notation and variables in network model and SFC model.

Notation	Description
$\mathcal{G}$	Set of edge gateways, $\mathcal{G} = \{1, 2, \dots, N_g\}$
$\mathcal{D}$	Set of IIoT devices, $\mathcal{D} = \{1, 2, \dots, N_d\}$
$\mathcal{V}$	Set of all computing nodes, $\mathcal{V} = \mathcal{G} \cup \mathcal{D}$
N_g	Number of edge gateways
N_d	Number of IIoT devices
C_i	Computing capacity of node i (CPU cycles per time slot)
F_i	CPU processing rate of node i (cycles/s)
S_i	Maximum storage capacity of node i
R_n	The n -th SFC request
$\mathcal{F}_n$	Ordered VNF sequence of SFC n , $\mathcal{F}_n = \{v_{n,1}, v_{n,2}, \dots, v_{n, \mathcal{F}_n }\}$
$v_{n,k}$	The k -th VNF of SFC request n
$\mathcal{C}_n$	Computational complexity vector of VNF, $\mathcal{C}_n = \{c_{n,1}, c_{n,2}, \dots, c_{n, \mathcal{F}_n }\}$
$c_{n,k}$	Computational complexity of VNF $v_{n,k}$ (CPU cycles/bit)
$\mathcal{S}_n$	Storage requirement vector of VNF, $\mathcal{S}_n = \{s_{n,1}, s_{n,2}, \dots, s_{n, \mathcal{F}_n }\}$
$s_{n,k}$	Storage requirement of VNF $v_{n,k}$ (including image and state data)
$\mathcal{L}_n$	Data packet length vector, $\mathcal{L}_n = \{l_{n,1}, l_{n,2}, \dots, l_{n, \mathcal{F}_n }\}$
$l_{n,0}$	Initial data packet length of SFC request n (bits)
$l_{n,k}$	Data packet length after processing by VNF $v_{n,k}$ (bits)
Δ_n	Maximum tolerable end-to-end delay of SFC n (ms)

2.3. Communication Model

The communication model is the basis for calculating the end-to-end delay and service cost. In the system of this paper, three key communication scenarios are considered: inter-gateway communication between edge gateways, gateway-to-device communication between edge gateways and IIoT devices, and local communication between IIoT devices.

First, the channel power gain $g_{i,j,t}$ between any two nodes i and j is defined as

$$g_{i,j,t} = \delta_{i,j} \cdot (d_{i,j,t})^{-\alpha} \quad \forall i, j \in \mathcal{V}, i \neq j, \quad (3)$$

where α is the path loss exponent, satisfying $\alpha \geq 2$, and $\delta_{i,j}$ represents the small-scale fading of the channel, following the Rayleigh fading model, satisfying $\delta_{i,j} \sim \exp(\beta)$. After establishing the channel model, the data transmission rate $r_{i,j,t}$ is derived according to the Shannon–Hartley theorem:

$$r_{i,j,t} = W_{i,j} \cdot \log_2 \left(1 + \frac{p^i \cdot g_{i,j,t}}{N_0} \right), \quad (4)$$

where $W_{i,j}$ is the channel bandwidth between nodes i and j , p^i is the transmission power of the transmitting node i , and N_0 is the noise power. Finally, the communication delay and energy consumption generated during the process of the SFC request $\mathcal{R}_n$ from $v_{n,k-1}$ to $v_{n,k}$ are modeled. First, when the computing tasks of VNF $v_{n,k}$ are distributed across multiple nodes, the scheduling ratio $\alpha_{n,k,i,t}$ fundamentally determines the fraction of the input traffic stream that node i is assigned to process. Thus, the actual input data $l_{n,k,i,t}^{\text{in}}$ allocated to node i can be expressed as

$$l_{n,k,i,t}^{\text{in}} = \alpha_{n,k,i,t} \cdot l_{n,k-1,t}^{\text{out}}. \quad (5)$$

However, not all nodes in the network participate in the calculation of $v_{n,k}$. Therefore, this paper introduces a dynamic set $\mathcal{V}_{n,k,t}$ to represent all nodes that actually execute $v_{n,k}$ in time slot t , expressed as

$$\mathcal{V}_{n,k,t} = \{i \in \mathcal{V} \mid z_{n,k,i,t} = 1\}, \quad (6)$$

Based on this set, to ensure data flow conservation, the total input data of VNF $v_{n,k}$ must satisfy the following relation:

$$l_{n,k}^{in} = \sum_{i \in \mathcal{V}_{n,k,t}} l_{n,k,i,t}^{in} \quad (7)$$

For notational simplicity, we omit the superscripts in and out in the subsequent formulas. Hereafter, variables indexed with k (e.g., $l_{n,k,i,t}$) denote the input data for computing, while variables indexed with $k-1$ (e.g., $l_{n,k-1}$) denote the output data for transmission. Based on the above definitions, in the inter-gateway link, the communication delay $T_{n,k,t}^{\text{gg-comm}}$ is expressed as

$$T_{n,k,t}^{\text{gg-comm}} = \max_{\substack{g \in \mathcal{G}_{n,k-1,t} \\ g' \in \mathcal{G}_{n,k,t}}} \left\{ \frac{l_{n,k-1,g,t}}{r_{g,g',t}} \right\}, \quad (8)$$

where $\mathcal{G}_{n,k,t}$ represents the dynamic set of edge gateway nodes that actually execute VNF $v_{n,k}$ at time slot t , and the max operator reflects the synchronization requirement arising from distributed computing offloading decisions; as the computational tasks of VNF $v_{n,k-1}$ may be distributed across multiple source nodes for collaborative processing through computing scheduling ratio $\alpha_{n,k-1,i,t}$, these distributed source nodes need to transmit their processing results to multiple destination nodes executing VNF $v_{n,k}$, thus ensuring that VNF $v_{n,k}$ must wait until all data from the slowest transmission link has been received before it can begin processing. Its communication energy consumption $E_{n,k,t}^{\text{gg-comm}}$ is expressed as

$$E_{n,k,t}^{\text{gg-comm}} = \sum_{\substack{g \in \mathcal{G}_{n,k-1,t} \\ g' \in \mathcal{G}_{n,k,t}}} p^g \cdot \frac{l_{n,k-1,g,t}}{r_{g,g',t}}, \quad (9)$$

where the summation operator reflects the cumulative nature of energy consumption, as all active transmission links consume energy simultaneously and independently during the communication process, making the total energy consumption the cumulative sum of individual link consumptions. In the gateway-to-device link, the communication delay $T_{n,k,t}^{\text{gd-comm}}$ is expressed as

$$T_{n,k,t}^{\text{gd-comm}} = \max_{\substack{g \in \mathcal{G}_{n,k-1,t} \\ d \in \mathcal{D}_{n,k,t}}} \left\{ \frac{l_{n,k-1,g,t}}{r_{g,d,t}} \right\}, \quad (10)$$

where $\mathcal{D}_{n,k,t}$ represents the dynamic set of IIoT device nodes that actually execute VNF $v_{n,k}$ at time slot t . Its total communication energy consumption $E_{n,k,t}^{\text{gd-comm}}$ is expressed as

$$E_{n,k,t}^{\text{gd-comm}} = \sum_{\substack{g \in \mathcal{G}_{n,k-1,t} \\ d \in \mathcal{D}_{n,k,t}}} p^g \cdot \frac{l_{n,k-1,g,t}}{r_{g,d,t}}. \quad (11)$$

Finally, for the ground link, its communication delay $T_{n,k,t}^{\text{dd-comm}}$ is expressed as

$$T_{n,k,t}^{\text{dd-comm}} = \max_{\substack{d \in \mathcal{D}_{n,k-1,t} \\ d' \in \mathcal{D}_{n,k,t}}} \left\{ \frac{l_{n,k-1,d,t}}{r_{d,d',t}} \right\}, \quad (12)$$

Its total communication energy consumption $E_{n,k,t}^{\text{dd-comm}}$ is expressed as

$$E_{n,k,t}^{\text{dd-comm}} = \sum_{\substack{d \in \mathcal{D}_{n,k-1,t} \\ d' \in \mathcal{D}_{n,k,t}}} p^d \cdot \frac{l_{n,k-1,d,t}}{r_{d,d',t}}. \quad (13)$$

2.4. Computational Model

This paper believes that the total computing delay $T_{n,k,t}^{\text{comp}}$ of a VNF consists of two consecutive stages: the VNF instantiation stage and the data processing stage. Assume VNF $v_{n,k}$ is instantiated in parallel on all selected target nodes. The completion time of the entire instantiation process depends on the slowest node. Therefore, the VNF instantiation delay $T_{n,k,t}^{\text{ins}}$ is defined as

$$T_{n,k,t}^{\text{ins}} = \max_{i \in \mathcal{V}} \left\{ z_{n,k,i,t} \cdot d_{i,k}^{\text{ins}} \right\}, \quad (14)$$

where $d_{i,k}^{\text{ins}}$ is the time required to instantiate VNF $v_{n,k}$ on node i . After instantiation, the data processing process is carried out. Since the computing tasks of the VNF are collaboratively assigned to multiple nodes for parallel processing, the total processing delay depends on the slowest node, as the output of $v_{n,k}$ can only be generated after all parallel subtasks have completed their computation. The VNF processing delay $T_{n,k,t}^{\text{proc}}$ is defined as

$$T_{n,k,t}^{\text{proc}} = \max_{i \in \mathcal{V}} \left\{ z_{n,k,i,t} \cdot \frac{c_{n,k} \cdot \alpha_{n,k,i,t} \cdot l_{n,k-1}}{F_i} \right\}. \quad (15)$$

The computing energy consumption generated during the processing of the SFC consists of two parts: the energy consumption $E_{n,k,t}^{\text{comp},\mathcal{D}}$ generated by the IIoT device and the energy consumption $E_{n,k,t}^{\text{comp},\mathcal{G}}$ generated by the edge gateway. The energy consumption calculation is based on the CMOS dynamic power consumption principle, where power consumption is proportional to the square of processor operating frequency, the computational complexity of tasks, and the amount of data processed, which are expressed as

$$E_{n,k,t}^{\text{comp},\mathcal{D}} = \sum_{d \in \mathcal{D}} z_{n,k,d,t} \cdot \kappa_d \cdot c_{n,k} \cdot l_{n,k,i,t} \cdot (F_d)^2, \quad (16)$$

$$E_{n,k,t}^{\text{comp},\mathcal{G}} = \sum_{g \in \mathcal{G}} z_{n,k,g,t} \cdot \kappa_g \cdot c_{n,k} \cdot l_{n,k,i,t} \cdot (F_g)^2, \quad (17)$$

where κ_i is the effective capacitance coefficient depending on the chip architecture used.

2.5. Problem Formulation

This paper aims to conduct a global joint optimization of the edge gateway-assisted edge computing network. The core objective is to co-optimize SFC orchestration and VNF deployment $\mathcal{Z} = \{z_{n,k,i,t}\}$ with computing scheduling ratio $\mathcal{A} = \{\alpha_{n,k,i,t}\}$ under dynamic service requests and network environments, so as to minimize the total operating cost of the system while ensuring QoS requirements. This total cost is a comprehensive weighted manifestation of the end-to-end service delay and the total system energy consumption. To this end, this paper first defines two key performance indicators of the system: the total delay and the total energy consumption. And we hereby declare two key parameters: N_R represents the total number of SFC requests in the current system, while K_n represents the length (i.e., the number of VNFs) of the n -th SFC request $\mathcal{R}_n$.

For any SFC request $\mathcal{R}_n$, its end-to-end total delay $T_{n,t}$ is the accumulation of the communication delay and computing delay of all VNFs on its link:

$$\begin{aligned} T_{n,t} &= \sum_{k=1}^{K_n} (T_{n,k,t}^{\text{comm}} + T_{n,k,t}^{\text{comp}}) \\ &= \sum_{k=1}^{K_n} (T_{n,k,t}^{\text{gg-comm}} + T_{n,k,t}^{\text{gd-comm}} + T_{n,k,t}^{\text{dd-comm}} \\ &\quad + T_{n,k,t}^{\text{ins}} + T_{n,k,t}^{\text{proc}}), \end{aligned} \quad (18)$$

The total energy consumption E_t of the system in time slot t is the sum of the communication energy consumption and the computing energy consumption generated during the processing of all SFC requests:

$$\begin{aligned} E_t &= \sum_{n=1}^{N_R} \sum_{k=1}^{K_n} (E_{n,k,t}^{\text{comm}} + E_{n,k,t}^{\text{comp}}) \\ &= \sum_{n=1}^{N_R} \sum_{k=1}^{K_n} (E_{n,k,t}^{\text{gg-comm}} + E_{n,k,t}^{\text{gd-comm}} + E_{n,k,t}^{\text{dd-comm}} \\ &\quad + E_{n,k,t}^{\text{comp},\mathcal{D}} + E_{n,k,t}^{\text{comp},\mathcal{G}}). \end{aligned} \quad (19)$$

The optimization objectives are as follows:

$$\min_{\mathcal{Z}, \mathcal{A}} \Phi = w_T \sum_{n=1}^{N_R} T_{n,t} + w_E E_t, \quad (20)$$

Here, $w_T + w_E = 1$, where w_T and w_E are adjustable weight factors for balancing latency and energy consumption. Higher w_T values prioritize delay optimization for time-sensitive applications (e.g., robotic control), while higher w_E values prioritize energy efficiency for battery-powered devices (e.g., remote sensors),

subject to

$$\text{C1: } \sum_{n=1}^{N_R} \sum_{k=1}^{K_n} \alpha_{n,k,i,t} \cdot c_{n,k} \cdot l_{n,k-1} \leq C_i \quad \forall i \in \mathcal{V}, \quad (21)$$

$$\text{C2: } \sum_{n=1}^{N_R} \sum_{k=1}^{K_n} z_{n,k,i,t} \cdot s_{n,k} \leq S_i \quad \forall i, t, \quad (22)$$

$$\text{C3: } \sum_{i \in \mathcal{V}} \alpha_{n,k,i,t} = 1 \quad \forall n, k, \quad (23)$$

$$\text{C4: } \alpha_{n,k,i,t} \leq z_{n,k,i,t} \quad \forall n, k, i, t, \quad (24)$$

$$\text{C5: } T_{n,t} \leq \Delta_n \quad \forall n, \quad (25)$$

$$\text{C6: } z_{n,k,i,t} \in \{0, 1\} \quad \forall n, k, i, \quad (26)$$

$$\text{C7: } \alpha_{n,k,i,t} \in [0, 1] \quad \forall n, k, i. \quad (27)$$

The optimization objective must be constrained by C1–C7 to ensure its effectiveness. C1 ensures that the computing load allocated to each node does not exceed its maximum computing capacity. C2 guarantees that the cumulative storage requirements of all VNFs deployed on any node, including their image footprint and dynamic state data, do not exceed its physical storage capacity. C3 guarantees that the sum of the proportions of the computing tasks of each VNF allocated on all nodes is 1. C4 ensures that only when a VNF is deployed to a certain node can computing tasks be allocated. C5 ensures that the

end-to-end delay of each service chain does not exceed its maximum tolerable delay. C6–C7 constrain the value ranges of the decision variables.

3. SORA-MAPPO Algorithm

The joint optimization problem in this paper involves multiple nodes making autonomous yet interdependent decisions on VNF deployment and resource allocation. Since each node can only access local observations, traditional centralized methods face severe challenges in such distributed scenarios. To this end, this paper adopts a MADRL framework, modeling each node as an independent learning agent, and proposes the SORA-MAPPO algorithm based on the CTDE paradigm, where global information guides training while each agent executes decisions independently based on local observations [28].

3.1. DEC-POMDP Formulation

The joint optimization problem constructed in this paper is a mixed-integer nonlinear programming (MINLP) problem, which has been proven to be NP-hard. However, traditional optimization methods struggle to effectively address such large-scale and highly dynamic scenarios due to their high computational complexity and reliance on accurate system models. To this end, this paper adopts DRL to solve the problem [29]. DRL, as a data-driven decision-making paradigm, learns through the interaction between agents and the environment to find optimal decisions suitable for such high-dimensional dynamic scenarios. Therefore, this paper models the original optimization problem as a Markov Decision Process (MDP) [30,31], characterized by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{R} \rangle$, where $\mathcal{S}$, $\mathcal{A}$, and $\mathcal{R}$ represent the state space, action space, and reward function, respectively.

State Space $\mathcal{S}_{i,t}$: In the multi-agent framework proposed in this paper, all available computing nodes i in the network are modeled as agents, and each agent only has partial observation ability to the global environment. At each time slot t , the local observation state $\mathcal{S}_{i,t}$ of agent i consists of three parts: its own state $\mathcal{S}_{i,t}^{\text{self}}$, neighborhood state $\mathcal{S}_{i,t}^{\text{neighbor}}$, and VNF state $\mathcal{S}_t^{\text{VNF}}$.

First, the own state $\mathcal{S}_{i,t}^{\text{self}}$ is defined as

$$\mathcal{S}_{i,t}^{\text{self}} = \{f_{i,t}, p_{i,t}, e_{i,t}\}, \quad (28)$$

This state reflects the inherent resources and physical constraints of agent i , including its remaining computing resources $f_{i,t}$, current location $p_{i,t}$ and remaining energy $e_{i,t}$. Second, the neighborhood state $\mathcal{S}_{i,t}^{\text{neighbor}}$ is defined as

$$\mathcal{S}_{i,t}^{\text{neighbor}} = \{f_{j,t}, W_{ij,t} | \forall j \in \mathcal{N}_{i,t}\}, \quad (29)$$

This state describes the interactive environment between the agent and its neighbors within communication range. In this paper, the set of neighborhood nodes of agent i at time slot t is denoted as $\mathcal{N}_{i,t}$, and this set contains all nodes j within its communication range. Therefore, it includes the remaining computing resources $f_{j,t}$ of each neighborhood node j and the available bandwidth $W_{ij,t}$ of the link connected to agent i . Finally, the VNF state $\mathcal{S}_t^{\text{VNF}}$ is defined as

$$\mathcal{S}_t^{\text{VNF}} = \{c_{n,k}, l_{n,k-1}, \Delta_{n,t} | \forall v_{n,k} \in \mathcal{V}_t\}, \quad (30)$$

This state is constructed as globally shared information reflecting the characteristics of pending service chain requests, and can be observed by each agent i . In addition, the set of all VNFs to be processed at time slot t is defined as $\mathcal{V}_t$. Therefore, it includes VNF $v_{n,k}$, the inherent computing complexity $c_{n,k}$, the required data input volume $l_{n,k-1}$, and the remaining delay tolerance $\Delta_{n,t}$ at time slot t .

Therefore, the state space $\mathcal{S}_{i,t}$ of each agent is represented as

$$\mathcal{S}_{i,t} = \{\mathcal{S}_{i,t}^{\text{self}}, \mathcal{S}_{i,t}^{\text{neighbor}}, \mathcal{S}_t^{\text{VNF}}\}. \quad (31)$$

Action Space $\mathcal{A}_i$: The action $a_{i,t}$ of agent i at each time slot t is a composite action. This action space $\mathcal{A}_i$ includes: VNF deployment decision variables $z_{i,t} = \{z_{n,k,i,t} \mid \forall v_{n,k} \in \mathcal{F}_n\}$, used to decide whether the k -th VNF $v_{n,k}$ of SFC request $\mathcal{R}_n$ is deployed on node i ; and computing resource scheduling ratio variables $\alpha_{i,t} = \{\alpha_{n,k,i,t} \mid \forall v_{n,k} \in \mathcal{F}_n\}$, which allocate computing power to deployed VNFs and finely quantify the proportion of computing power contribution of node i to this VNF, thus supporting cross-node collaborative computing. Therefore, the action space of agent i at time slot t can be expressed as

$$a_{i,t} = \{z_{i,t}, \alpha_{i,t}\} \quad a_{i,t} \in \mathcal{A}_i. \quad (32)$$

To effectively handle this hybrid action space comprising both discrete and continuous variables without artificial discretization, the Actor network of each agent is designed with a multi-head output architecture. Specifically, a Categorical (Softmax) head is utilized to output the discrete VNF deployment decision $z_{n,k,i,t}$, while a parallel Gaussian or Beta distribution head simultaneously generates the continuous resource scheduling ratio $\alpha_{n,k,i,t}$. Both types of actions are generated in a single forward pass, which effectively prevents the precision loss and action-space explosion typically caused by discretization in traditional value-based methods.

Reward Function $\mathcal{R}_t$: The reward function is the core feedback signal provided by the environment to agents, and its design must be consistent with the objective function of the original optimization problem. Therefore, we define the immediate reward $\mathcal{R}_t$ at each time slot t as a composite function consisting of two parts: one part directly reflects the optimization objective, and the other part is a penalty term P_t introduced to ensure that key system constraints are satisfied, specifically expressed as

$$\mathcal{R}_t = -(w_T \sum_{n=1}^{N_R} T_{n,t} + w_E E_t) - P_t. \quad (33)$$

The penalty term P_t is designed to enforce various system constraints:

$$\begin{aligned} P_t = & \lambda_1 \sum_{i \in \mathcal{V}} \max\{0, \sum_{n=1}^{N_R} \sum_{k=1}^{K_n} (\alpha_{n,k,i,t} \cdot c_{n,k} \cdot l_{n,k-1}) - C_i\} \\ & + \lambda_2 \sum_{n=1}^{N_R} \sum_{k=1}^{K_n} (\sum_{i \in \mathcal{V}} \alpha_{n,k,i,t} - 1)^2 \\ & + \lambda_3 \sum_{n=1}^{N_R} \max\{0, T_{n,t} - \Delta_n\}, \end{aligned} \quad (34)$$

where λ_1 to λ_3 are penalty coefficients for different constraint violations. In addition to these weighted penalties, the strict logical constraint (C4) is enforced independently via a large constant penalty. This strong negative reinforcement effectively prevents the agent from allocating traffic to non-deployed VNFs.

3.2. MAPPO Algorithm Framework

To learn an optimal action policy in the dynamic environment, this paper constructs a decision-making framework based on SORA-MAPPO. Within this framework, each agent utilizes the observed state vector as a common input for its Actor and Critic networks, thereby enabling environment-aware intelligent decision-making. As shown in Figure 3,

the framework adheres to the CTDE paradigm, aiming to achieve efficient collaboration among heterogeneous agents in the edge gateway-assisted IIoT network.

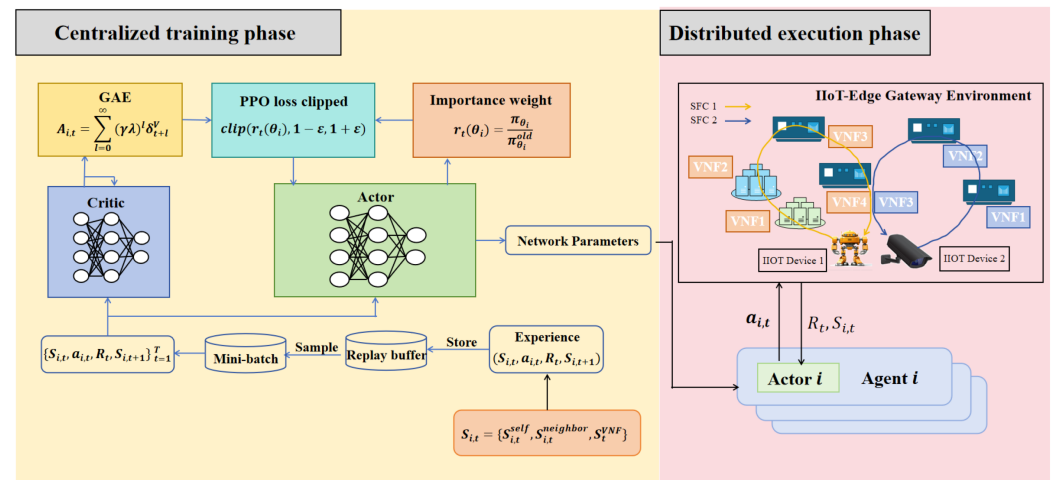


Figure 3. SORA-MAPPO framework.

MAPPO is a multi-agent DRL algorithm based on the Actor–Critic framework. In MAPPO, each agent has an independent Actor network, and all agents share a central Critic network during the training phase to evaluate the agents' actions. The Actor network is defined by parameters θ_i , and the Critic network is defined by parameters μ_i . The loss function of the Actor network in MAPPO is defined as

$$J(\theta_i) = E_t[\min\{\varphi_t(\theta_i)A_{i,t}, \text{clip}(\varphi_t(\theta_i), 1 - \varepsilon, 1 + \varepsilon)A_{i,t}\}], \quad (35)$$

Here, the function $\text{clip}(r_t(\theta_i), 1 - \varepsilon, 1 + \varepsilon)$ restricts $r_t(\theta_i)$ within the range of $[1 - \varepsilon, 1 + \varepsilon]$. $r_t(\theta_i)$ is the importance sampling ratio for training, defined as

$$\varphi_t(\theta_i) = \frac{\pi_{\theta_i}\{a_{i,t}|S_{i,t}\}}{\pi_{\theta_i^{\text{old}}}\{a_{i,t}|S_{i,t}\}}, \quad (36)$$

Here, $\pi_{\theta_i}\{a_{i,t}|S_{i,t}\}$ is generated by the real-time interaction between the agent and the environment, and $\pi_{\theta_i^{\text{old}}}\{a_{i,t}|S_{i,t}\}$ is the old policy for the interaction between the agent and the environment, which can be sampled from the historical data in the experience pool. Using the importance sampling ratio, the interaction information under the old policy can be used to estimate the new policy, thereby optimizing the new policy and accelerating convergence.

In MAPPO, Generalized Advantage Estimation (GAE) is introduced to estimate the advantage function $A_{i,t}$. The advantage function can effectively reduce the variance of the estimation, and its definition is

$$A_{i,t} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V, \quad (37)$$

Here, γ represents the discount factor, $\gamma \in [0, 1]$. λ are the GAE hyperparameters, $\lambda \in [0, 1]$. δ_{t+l}^V represents the temporal difference error at $t + l$ time and $t + l + 1$ time.

$$\delta_{t+l}^V = R_{t+l} + \gamma V_{\mu_i}(S_{t+l+1}) - V_{\mu_i}(S_{t+l}), \quad (38)$$

Here, V_{μ_i} state value function. In addition, the loss function of the Critic network is defined as

$$J(\mu_i) = E_t[(R_t + \gamma V_{\mu_i}(S_{t+1}) - V_{\mu_i}(S_t))^2]. \quad (39)$$

Furthermore, benefiting from the CTDE architecture, SORA-MAPPO achieves zero additional communication overhead during the actual deployment. The information exchange in this framework primarily occurs when agents share observations with the centralized Critic network. However, this centralized training phase is strictly conducted offline on high-performance servers, thus consuming no real-time network bandwidth. During the online application phase, the system operates via fully distributed execution. Agents infer actions locally based on their own observations without any real-time coordination messages. Therefore, the proposed method introduces no extra communication overhead to the live industrial environment, further guaranteeing the ultra-low latency requirements of IIoT services.

3.3. Computational Complexity Analysis

Following [32,33], we systematically evaluate the computational complexity of the proposed SORA-MAPPO algorithm. Based on the CTDE paradigm, the total computational overhead primarily depends on the multi-layer perceptron (MLP) architectures of the Actor and Critic networks, as well as the training horizon.

Let L_a and L_c denote the total number of fully connected (FC) layers in the Actor network and the Critic network, respectively. Similarly, let u_j^a and u_j^c represent the number of neurons in the j -th layer of the corresponding Actor and Critic networks. Specifically, the output dimension of the Actor network is determined by the hybrid action space, which includes the discrete VNF deployment decision $z_{n,k,i,t}$ and continuous computing scheduling ratio $\alpha_{n,k,i,t}$. For standard FC networks, the computational complexity of a single forward or backward propagation through the j -th layer is proportional to the product of the input and output dimensions, i.e., $\mathcal{O}(u_{j-1}u_j)$.

(1) Complexity in the Training Phase: During the centralized training phase, as outlined in Algorithm 1, a total of $|V|$ nodes continually interact with the IIoT environment. The training process spans L episodes, with each episode containing T time steps. In each step, every agent feeds its local observation into the Actor network to generate a policy, while the shared Critic network gathers global states to evaluate the action-value function and compute gradient updates. Consequently, the overall computational complexity during the training phase can be formulated as

$$\mathcal{O}\left(|V| \cdot L \cdot T \cdot \left(\sum_{j=1}^{L_a} u_{j-1}^a u_j^a + \sum_{j=1}^{L_c} u_{j-1}^c u_j^c\right)\right). \quad (40)$$

(2) Complexity in the Execution Phase: Once the training process converges, the system transitions to the decentralized execution phase. In this stage, the centralized Critic network is completely discarded. Each node makes real-time SFC orchestration decisions solely by performing a forward pass through its locally deployed Actor network. Therefore, for a complete operational cycle of T steps, the execution complexity for the multi-agent system is drastically reduced to

$$\mathcal{O}\left(|V| \cdot T \cdot \sum_{j=1}^{L_a} u_{j-1}^a u_j^a\right). \quad (41)$$

Algorithm 1 Training phase of SORA-MAPPO

```

1: Initialize  $\theta_i$ ,  $\mu_i$  and buffer  $D$ .
2: for each episode = 1, 2, ...,  $L$  do
3:   for each agent  $i$  do
4:     Initialize state  $S_{i,1}$  and let  $\pi_{\theta_i}^{\text{old}} \leftarrow \pi_{\theta_i}$ 
5:   end for
6:   for each time step  $t = 1, 2, \dots, T$  do
7:     for each agent  $i$  do
8:       Obtain  $S_{i,t}$  from the environment
9:       Takes action  $a_{i,t}$  based on state  $S_{i,t}$  according to the policy  $\pi_{\theta_i}^{\text{old}}$ 
10:      Renew state  $S_{i,t+1}$ 
11:    end for
12:    Obtain a global instant reward  $R_t$  according to (33)
13:    for each agent  $i$  do
14:      Renew state according to (31)
15:      Store  $(S_{i,t}, a_{i,t}, R_t, S_{i,t+1})$  in buffer  $D$ 
16:    end for
17:    if buffer  $D$  is full then
18:      Uniformly sample mini-batches from  $D$ 
19:      Calculate the GAE  $\{A_{i,t}^t\}_{t=1}^T$  according to (37)
20:      Update  $\theta_i$  by Actor network loss function (35)
21:      Update  $\mu_i$  by Critic network's loss function (39)
22:      Empty  $D$ 
23:    end if
24:  end for
25: end for

```

4. Simulation Results

4.1. Simulation Setup

To evaluate the performance of our proposed algorithm, we developed a simulator on a computer running the 64-bit Windows 11 operating system. The machine is equipped with a 13th Gen Intel® Core™ i7-13700H CPU, 16 GB of RAM, and an NVIDIA GeForce RTX 4060 Laptop GPU. All algorithms were implemented in Python 3.9 using the PyTorch 2.5.1 framework. The main configurations are listed in Table 2.

To comprehensively evaluate the performance of our proposed SORA-MAPPO algorithm, we selected four representative algorithms as benchmarks. These algorithms range from fundamental multi-agent reinforcement learning frameworks to classic heuristic methods, aiming to validate the effectiveness and advancement of our algorithm from multiple dimensions.

(1) MAAC (Multi-Agent Actor–Critic): An MADRL algorithm that incorporates a soft attention mechanism. In this algorithm, each agent's critic network can selectively attend to information from other agents based on context, thereby achieving more effective credit assignment. It represents another mainstream approach to MADRL-based collaboration [34].

(2) DQN (Deep Q-Network): A typical single-agent DRL algorithm. It combines Q-learning with deep neural networks to handle high-dimensional state spaces, learning the optimal VNF deployment policy through experience replay to maximize long-term rewards.

(3) PSO (Particle Swarm Optimization): A classic swarm intelligence algorithm. It simulates bird flock behavior, where particles iteratively update their positions based on personal- and global-best experiences to find a near-optimal VNF deployment strategy [35].

(4) Greedy: A classic heuristic algorithm. At each decision point, this algorithm deploys each VNF to the node that minimizes its immediate cost, without considering the impact of decisions on future states and long-term rewards.

Table 2. Constant parameters for experiments.

Parameter	Description	Value
N_g	Number of edge gateways	4
N_d	Number of IIoT devices	6
W_{gg}	Gateway-to-gateway channel bandwidth	20 MHz
W_{gd}	Gateway-to-device channel bandwidth	10 MHz
W_{dd}	Device-to-device channel bandwidth	5 MHz
p_i	Transmission power	0.1 W
N_0	Noise power	1×10^{-13} W
C_g	Gateway computational capacity (per timeslot)	4.4×10^{10} cycles
C_d	Device computational capacity (per timeslot)	2.2×10^{10} cycles
F_g	Gateway CPU processing rate	3.0×10^9 cycles/s
F_d	Device CPU processing rate	1.5×10^9 cycles/s
S_g	Gateway maximum storage capacity	50 GB
S_d	Device maximum storage capacity	8 GB
κ_g	Gateway energy coefficient	1×10^{-27}
κ_d	Device energy coefficient	1×10^{-28}
N_R	Number of SFC requests	3
$ F_n $	VNF sequence length	[3, 4]
$l_{n,0}$	Initial data size	[500, 1000] KB
$c_{n,k}$	VNF computational complexity	[1000, 5000] cycles/bit
$s_{n,k}$	VNF storage requirement	[100, 500] MB
$d_{i,k}^{\text{ins}}$	VNF instantiation time	[0.001, 0.005] s
Δ_n^1	Max end-to-end delay (delay-sensitive)	[20, 40] s
Δ_n^0	Max end-to-end delay (computation-intensive)	[40, 60] s

To quantitatively assess the performance of the algorithms, we employ the following key metrics:

(1) Cumulative Reward: The sum of rewards per episode, directly reflecting the learning efficiency and the quality of the final policy.

(2) Total End-to-End Delay: Calculated according to Equation (18) to evaluate the system's QoS and real-time performance.

(3) Total System Energy Consumption: Calculated based on Equation (19) to assess the operational cost and resource efficiency of the system.

(4) VNF Deployment Rate: The ratio of successfully deployed VNF instances to total VNF demands. Specifically, a VNF is considered successfully deployed only when it is allocated to a physical node that strictly satisfies both the computing capacity constraint (C1) and the storage capacity constraint (C2). This metric fundamentally measures the system's resource utilization and deployment capability under multidimensional resource limitations.

(5) QoS Satisfaction Rate: The proportion of SFC requests completed within the maximum tolerated delay (Δ_n), measuring the system's ability to guarantee delay commitments.

To ensure statistical reliability, all experiments are evaluated across multiple independent random seeds. Convergence curves display moving averages (solid lines) with variance (shaded regions), while performance comparison points represent averages over multiple trials to eliminate random noise.

4.2. Experimental Analysis

In Figure 4a, SORA-MAPPO demonstrates superior learning efficiency by achieving the highest cumulative reward with the fastest convergence rate. It reaches a stable state at episode 194, approximately $2.07\times$ faster than MAAC, which requires 402 episodes. In terms of practical overhead, SORA-MAPPO reduces the training duration to 8.08 min (compared to 17.11 min for MAAC), benefiting from the PPO clipping and GAE mechanisms that effectively stabilize policy updates. While DQN exhibits the earliest convergence around episode 48, it becomes trapped in a significantly lower reward plateau, reflecting its limited capacity to handle high-dimensional multi-agent coordination. In contrast, the heuristic-based PSO algorithm suffers from significant oscillations and poor stability, struggling to adapt to the highly dynamic IIoT environment. These learning trends are further reflected in the energy consumption curves in Figure 4b. SORA-MAPPO maintains the lowest energy footprint through optimized scheduling, whereas PSO and DQN result in higher overheads due to sub-optimal decision-making. Greedy performs the worst in both metrics due to the lack of long-term strategic planning. Overall, SORA-MAPPO achieves the most robust balance between system utility and energy conservation.

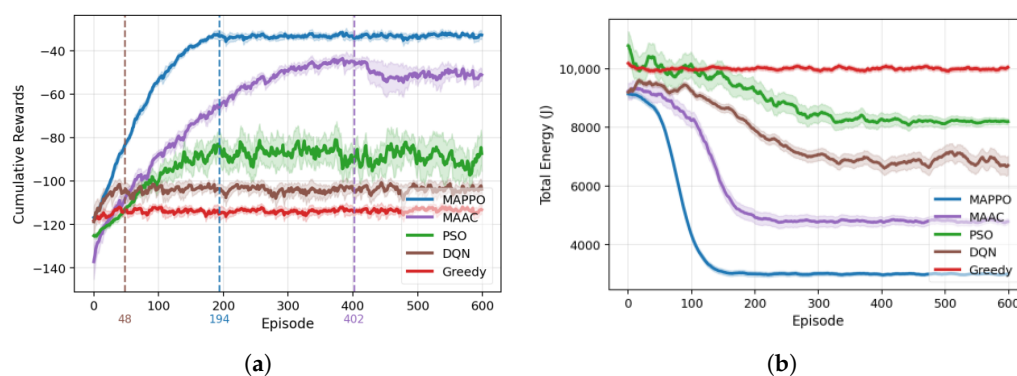


Figure 4. Convergence curves of different algorithms. (a) Cumulative reward convergence. (b) System energy consumption convergence.

Figure 5a investigates the impact of the actor learning rate on convergence performance. The results show that a large learning rate of 5×10^{-4} causes severe gradient oscillations during training and degrades the final performance, while a small learning rate of 5×10^{-5} converges too slowly and becomes trapped in a local optimum. Therefore, a learning rate of 2×10^{-4} achieves the best balance between convergence speed and stability, yielding the highest and most robust cumulative rewards. Figure 5b demonstrates the effect of batch size on performance. Small batch sizes (e.g., 32 and 64) lead to severe training instability and massive reward drops due to excessive gradient estimation variance, while an excessively large batch size (e.g., 192) causes training fluctuations and traps the policy in a sub-optimal state. Experiments prove that a batch size of 96 provides the most robust gradient updates, achieving the optimal balance between sample efficiency and stability, thereby guiding the algorithm to learn the most effective policy.

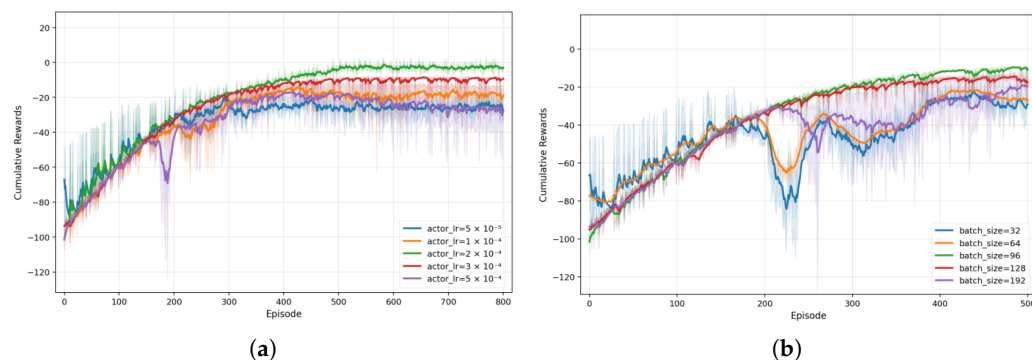


Figure 5. Impact of hyperparameters on training performance. (a) Effect of learning rate. (b) Effect of batch size.

To evaluate the scalability and robustness of SORA-MAPPO, we conducted experiments with varying numbers of edge gateways (N_g) and IIoT devices (N_d). Figure 6a shows the convergence as N_g increases from 4 to 10. Although a larger N_g expands the state–action space and increases initial exploration difficulty, the CTDE architecture enables agents to effectively learn cooperative strategies, eventually converging to a comparable optimal reward level. This demonstrates the algorithm’s excellent scalability in managing distributed edge resources. Figure 6b illustrates system performance under heavier traffic loads, with N_d scaling from 10 to 20. More devices generate concurrent SFC requests, intensifying resource competition and lowering the overall reward baseline. Despite this, SORA-MAPPO maintains stable convergence. These results validate the strong robustness and adaptability of the proposed scheme in large-scale IIoT networks.

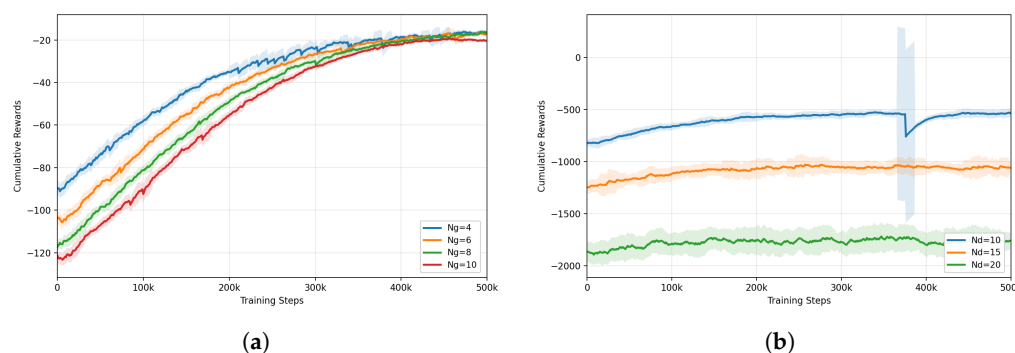


Figure 6. Robustness and scalability analysis of SORA-MAPPO. (a) Convergence under varying numbers of edge gateways (N_g). (b) Convergence under varying numbers of IIoT devices (N_d).

Figure 7 illustrates the impact of different penalty coefficient combinations ($\lambda_1, \lambda_2, \lambda_3$) defined in Equation (33) on the convergence performance of the SORA-MAPPO algorithm. Empirical results demonstrate that the configuration $(\lambda_1, \lambda_2, \lambda_3) = (1, 10, 1)$ yields the highest stable cumulative reward, accompanied by the fastest convergence rate and minimal variance. Conversely, sub-optimal parameter settings induce severe training oscillations and trap the policy optimization in local optima. This instability arises from an imbalanced penalty mechanism that disproportionately penalizes specific constraints, thereby hindering effective environmental exploration. Consequently, the combination $(1, 10, 1)$ is adopted in our experiments to guarantee a stable and optimal joint orchestration policy.

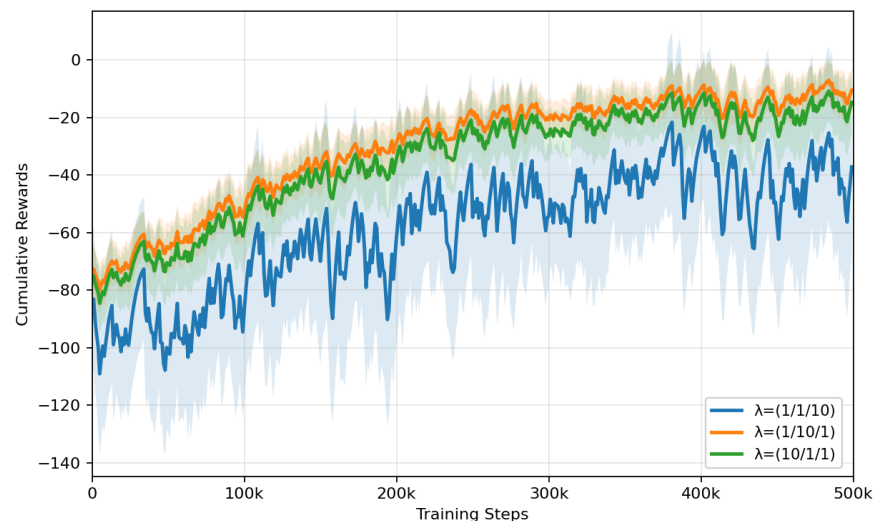


Figure 7. Convergence analysis under different penalty parameter configurations.

It is noteworthy that while DQN is evaluated in the convergence analysis, it is excluded from the subsequent performance comparison evaluations. As illustrated in the training results, although SORA-MAPPO efficiently converges to a high reward, DQN fails to learn an effective policy, performing only marginally better than the Greedy approach. The fundamental reason is that our formulated MINLP problem features a massive, hybrid action space (discrete VNF deployment and continuous computational scheduling). DQN, as a value-based method, requires artificial discretization of continuous variables, which triggers a severe “curse of dimensionality.” Consequently, we exclude DQN from performance comparisons to focus on benchmarks that are structurally capable of handling this complexity, such as MAAC and heuristic methods.

In Figure 8a, increasing node storage capacity improves the deployment rate, as larger storage space allows nodes to accommodate more VNF instances. MAPPO consistently leads because its CTDE framework enables forward-looking planning for the entire SFC chain. In contrast, PSO and Greedy fail to capture complex chain dependencies and only achieve sub-optimal results. In Figure 8b, increasing data size leads to a decline in deployment rates. MAPPO demonstrates the strongest robustness, maintaining a stable rate until 800 KB. However, MAAC’s performance plummets at 800 KB (from 0.9 to 0.35), and Greedy’s rate drops to zero at 900 KB. Although PSO remains non-zero, it stays at an extremely low level, reflecting the inability of these baseline models to handle high transmission overhead. In Figure 8c, more SFC requests intensify resource competition, reducing overall performance. MAPPO achieves efficient load balancing via multi-agent collaboration driven by a shared Critic. Conversely, MAAC and PSO exhibit lower efficiency due to the lack of centralized global evaluation or effective exploration mechanisms. In Figure 8d, MAPPO demonstrates excellent scalability, as more edge gateways provide richer deployment options. MAPPO fully exploits this expansion to optimize VNF placement. In contrast, Greedy and PSO show negligible benefits, proving they cannot efficiently navigate larger solution spaces. In Figure 8e, higher CPU frequency reduces VNF processing latency, enabling more instances to meet delay constraints. MAPPO steadily scales from 0.41 at 2.0 GHz to 0.76 at 4.0 GHz. While MAAC, PSO, and Greedy also show upward trends, their improvements are significantly limited by the structural flaws in their resource-scheduling strategies.

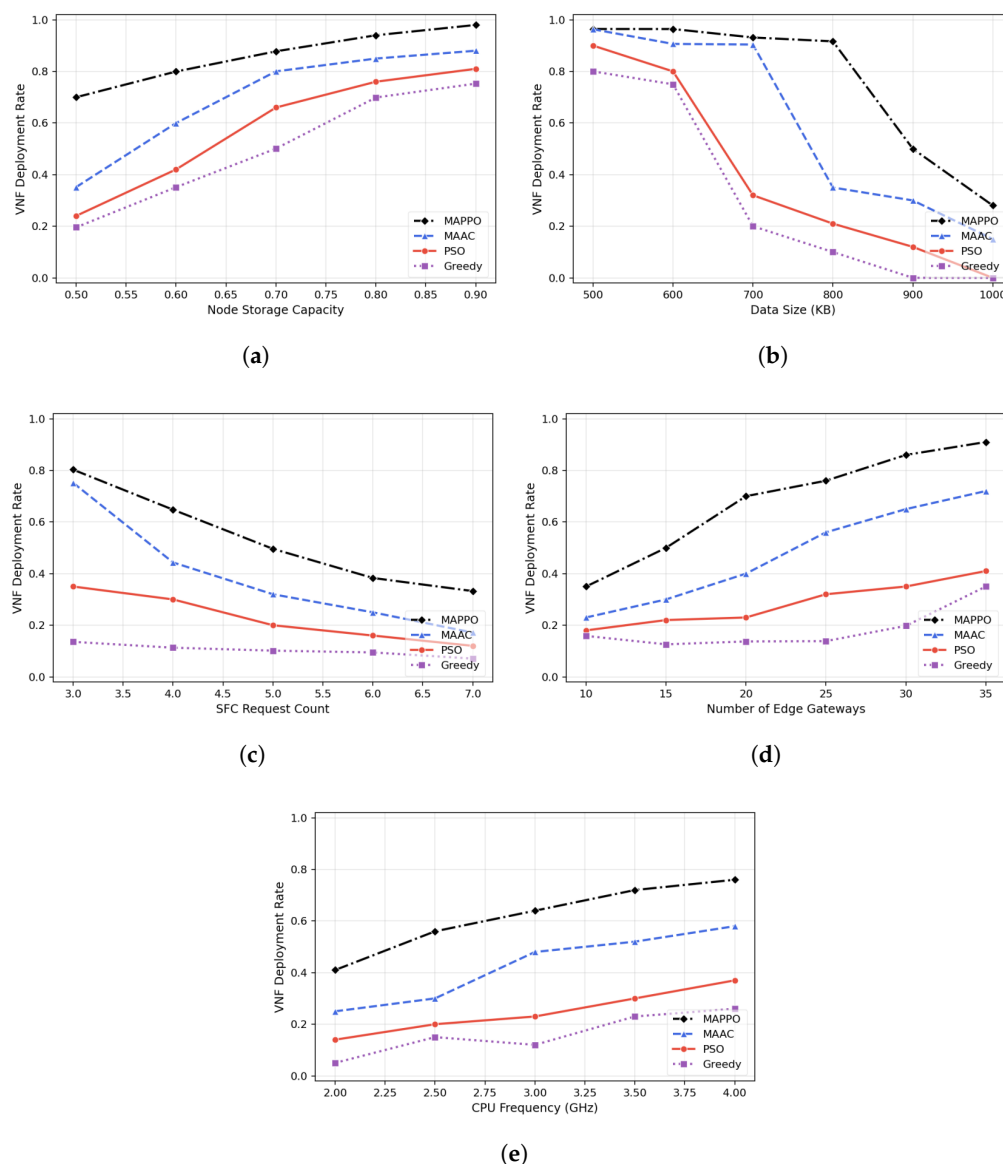


Figure 8. Impact of system parameters on VNF deployment rate. (a) Effect of node storage capacity. (b) Effect of data size. (c) Effect of SFC request count. (d) Effect of device number. (e) Effect of CPU frequency.

In Figure 9a, increasing data size leads to latency growth for all algorithms. MAPPO maintains the lowest latency across the entire range through joint optimization of VNF deployment and resource allocation. Greedy experiences a sharp latency surge to the order of 10^5 after 600 KB, indicating that its fixed strategy completely fails under high load. MAAC remains relatively stable before 700 KB but deteriorates sharply thereafter. PSO performs better than Greedy but stays significantly higher than MAPPO, as its heuristic search struggles to find optimal solutions in the expanded transmission-intensive space. In Figure 9b, increasing the number of SFC requests causes latency growth due to intensified resource contention. MAPPO's multi-agent collaboration mechanism enables more intelligent resource allocation, with latency smoothly increasing from approximately 3000 ms at three requests to about 17,000 ms at seven requests. Greedy consistently maintains extremely high latency at the order of 10^5 with almost no variation. MAAC's latency is consistently about twice that of MAPPO, while PSO fails to effectively coordinate resources across multiple requests, resulting in substantial delays. In Figure 9c, increasing the number

of edge gateways reduces system latency, as more gateways distribute computational load and provide shorter transmission paths. MAPPO efficiently explores the expanded decision space, reducing latency from approximately 30,000 ms at 10 gateways to about 2000 ms at 35 gateways, a 93% reduction. Although MAAC and PSO also show latency reduction, PSO's improvement is limited by its lower exploration efficiency in high-dimensional spaces compared to DRL-based methods. In Figure 9d, higher CPU frequency significantly reduces system latency by accelerating VNF processing. MAPPO's latency decreases from approximately 25,000 ms at 2.0 GHz to about 5000 ms at 4.0 GHz, an 80% reduction. Greedy maintains extremely high latency at the order of 10^5 with minimal improvement. While MAAC and PSO show moderate reduction, their performance remains substantially inferior to MAPPO, reflecting the structural limitations of their strategies in fully exploiting hardware upgrades.

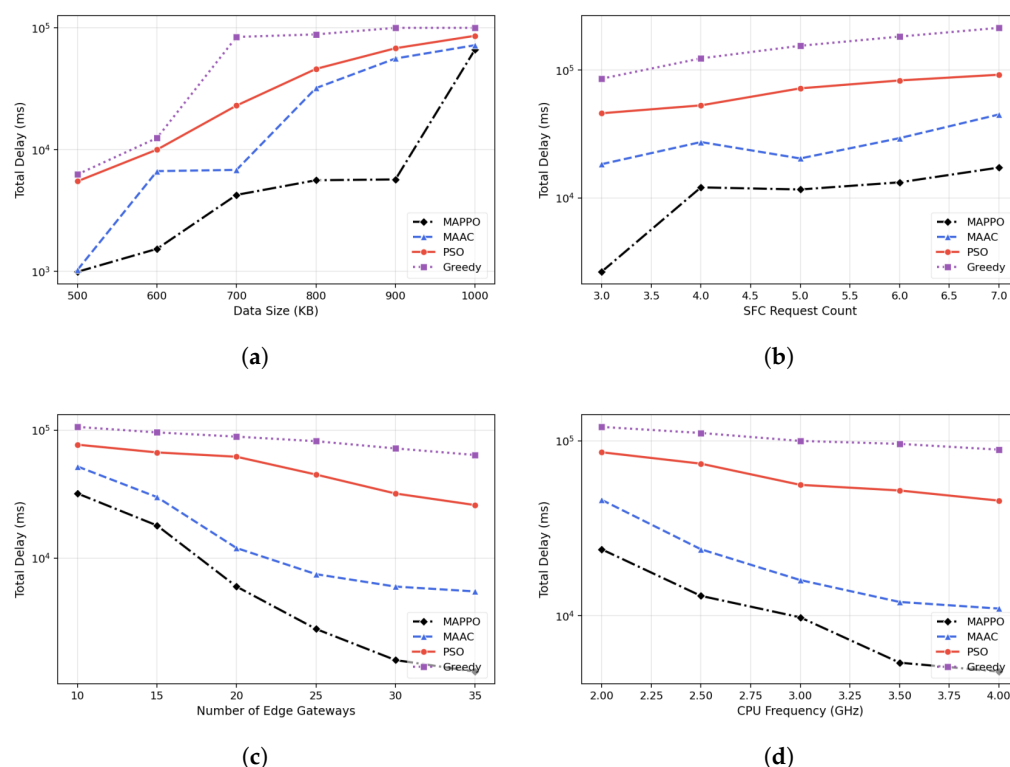


Figure 9. Impact of system parameters on total delay. (a) Effect of data size. (b) Effect of SFC request count. (c) Effect of device number. (d) Effect of CPU frequency.

In Figure 10a, increasing data size reduces QoS satisfaction due to heavier transmission loads. MAPPO more effectively balances resource demands, declining from 0.8 at 500 KB to approximately 0.45 at 900 KB. Greedy's QoS collapses to nearly zero at 700 KB, while MAAC and PSO also experience significant degradation, dropping to 0.15 and 0.05 respectively at 900 KB. In Figure 10b, increasing SFC requests reduces QoS due to intensified resource competition. MAPPO's centralized training enables superior request prioritization, declining from 0.8 at three requests to 0.35 at seven requests while consistently maintaining the highest satisfaction. Greedy drops to 0 after five requests, MAAC remains at approximately half of MAPPO's level, and PSO degrades rapidly from 0.3 to near zero, indicating structural limitations under severe contention. In Figure 10c, more edge gateways improve QoS by providing additional deployment options. MAPPO quickly adapts to topology changes, improving from 0.45 at 10 gateways to an impressive 0.85 satisfaction at 35 gateways. Greedy shows almost no variation until large-scale deployments. While MAAC and PSO improve, they max out at around 0.65 and 0.42 respectively, falling short

of MAPPO due to inadequate exploration of the enlarged solution space. In Figure 10d, higher CPU frequency improves QoS by accelerating VNF processing. MAPPO capitalizes most effectively on enhanced hardware, increasing from 0.44 at 2.0 GHz to 0.84 at 4.0 GHz. Greedy remains largely unresponsive below 0.2, and although MAAC and PSO show moderate improvements reaching 0.62 and 0.42 respectively, their decentralized or heuristic natures prevent them from fully exploiting the augmented resources like MAPPO.

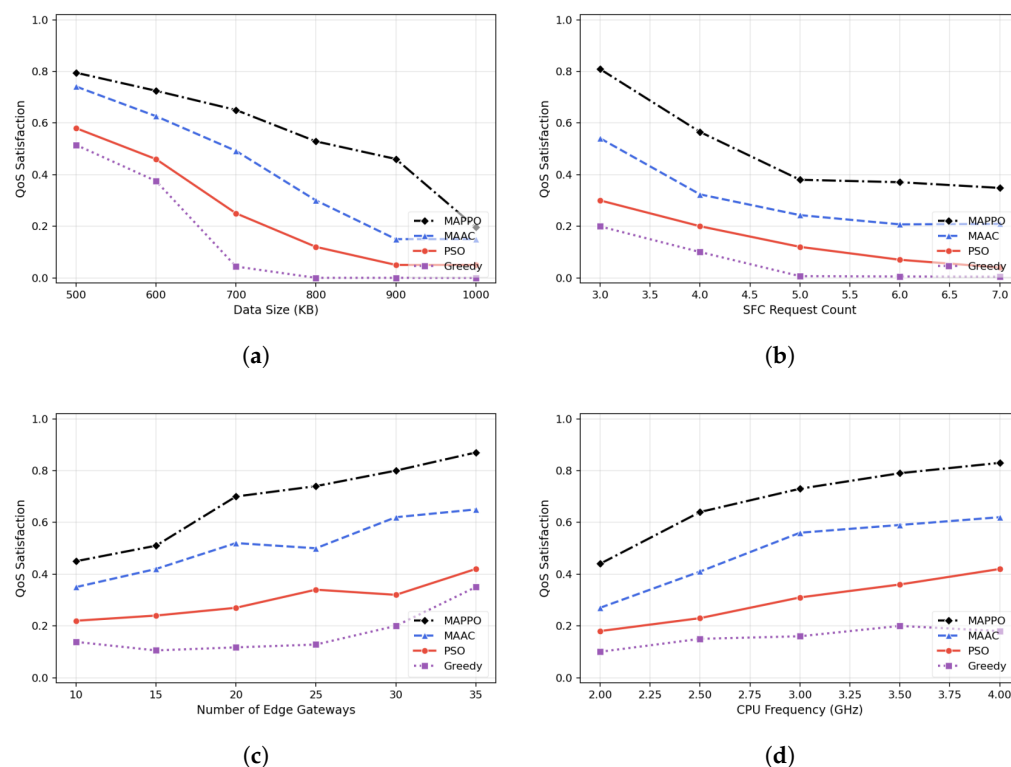


Figure 10. Impact of system parameters on QoS satisfaction. (a) Effect of data size. (b) Effect of SFC request count. (c) Effect of device number. (d) Effect of CPU frequency.

To further validate the necessity of the proposed joint-optimization framework, we conducted an ablation study comparing the joint-optimization scheme with two decoupled baselines: Fixed Compute (fixed $\alpha_{n,k,i,t}$) and Fixed VNF (fixed $z_{n,k,i,t}$). As illustrated in Figure 11a, the VNF deployment rate for all schemes declines as the number of SFC requests increases, due to the stringent resource limits of edge nodes. The joint-optimization scheme exhibits the highest deployment capability by dynamically coordinating $z_{n,k,i,t}$ and $\alpha_{n,k,i,t}$ to maximize resource utilization. In contrast, the performance of the Fixed-Compute scheme drops most sharply, as its inability to adjust the computing scheduling ratio leads to frequent violations of the computing capacity constraint C1. Correspondingly, Figure 11b demonstrates that the total delay grows exponentially under heavy traffic loads. According to Equation (15), processing delay is determined by the joint effect of VNF placement and computing allocation. The joint optimization effectively suppresses the acceleration of latency by fine-tuning the load distribution among heterogeneous nodes. Finally, Figure 11c shows the impact on QoS satisfaction. The decoupled schemes lack the flexibility to handle node congestion, easily leading to task timeouts. In contrast, the joint scheme intelligently circumvents congested nodes and allocates sufficient computing resources to urgent tasks, thereby guaranteeing the highest QoS level. These results explicitly prove that the deep coupling of SFC orchestration and resource scheduling is essential. By enabling agents to synergistically evaluate node capacities and network topologies, the joint optimization effectively prevents local resource bottlenecks and latency accumulation.

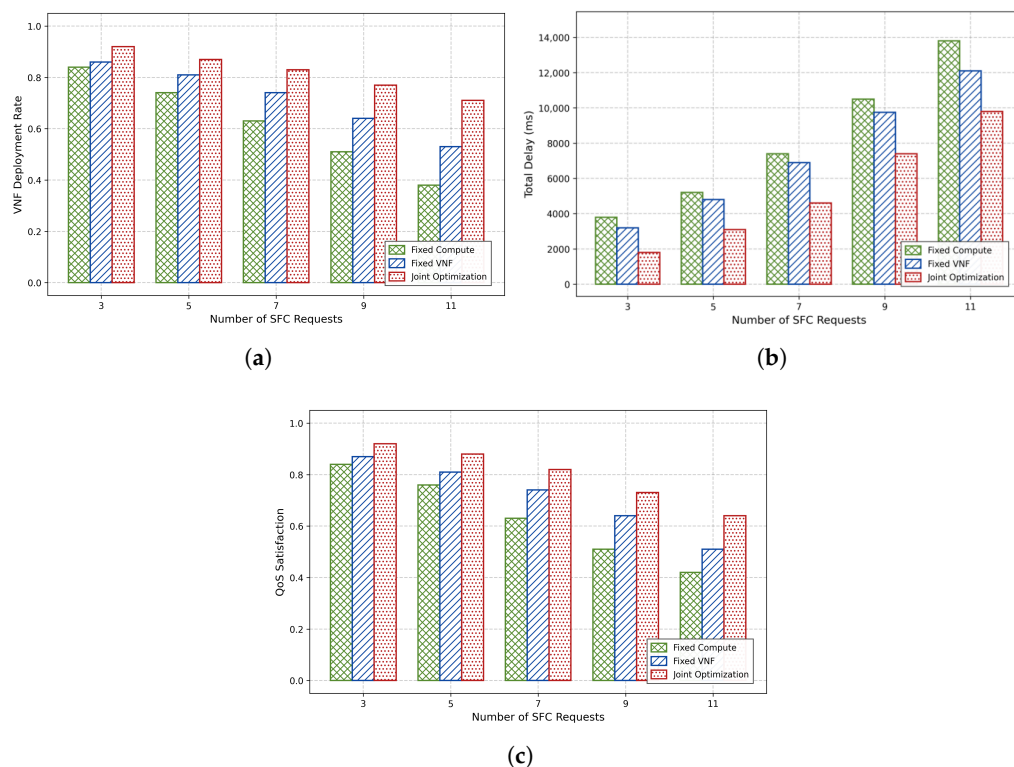


Figure 11. Ablation study of different optimization schemes. (a) VNF deployment rate. (b) Total delay. (c) QoS satisfaction.

5. Conclusions

To address the challenges posed by massive heterogeneous data streams and stringent low-latency requirements in the Industrial Internet of Things, this paper tackles the joint-optimization problem of Service Function Chain orchestration and resource allocation in edge gateway-assisted IIoT networks. The formulated MINLP model aims to minimize end-to-end latency and system energy consumption through optimizing VNF deployment decisions $z_{n,k,i,t}$ and resource scheduling ratios $\alpha_{n,k,i,t}$ for joint decision-making. The proposed SORA-MAPPO algorithm adopts the CTDE paradigm to model each node as a collaborative agent. Simulation results demonstrate that compared to the MAAC algorithm, the proposed approach achieves improvements of 20%, 42%, and 45% in VNF deployment rate, end-to-end latency, and QoS satisfaction rate, respectively, validating the effectiveness of the proposed scheme.

Author Contributions: Conceptualization, T.Z., L.W. and B.T.; methodology, T.Z., W.M. and B.T.; software, T.Z. and L.W.; validation, T.Z. and B.T.; formal analysis, T.Z. and B.W.; writing—original draft preparation, T.Z., W.M. and B.W.; writing—review and editing, T.Z. and B.T.; visualization, T.Z. and L.W.; supervision, B.T., W.M. and B.W. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported in part by the China Mobile Research Institute and in part by the National Science and Technology Major Project of China (Grant No. 2026ZD1307700).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: B.T., L.W., W.M., and B.W. are employed by the company China Mobile Research Institute. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Peralta, G.; Iglesias-Urkia, M.; Barcelo, M.; Gomez, R.; Moran, A.; Bilbao, J. Fog computing based efficient IoT scheme for the Industry 4.0. In Proceedings of the 2017 IEEE International Workshop of Electronics, Control, Measurement, Signals and their Application to Mechatronics (ECMSM), Donostia, Spain, 24–26 May 2017; pp. 1–6. [\[CrossRef\]](#)
2. Ning, Z.; Dong, P.; Wang, X.; Hu, X.; Guo, L.; Hu, B.; Guo, Y.; Qiu, T.; Leung, V.C.M. When Deep Reinforcement Learning Meets 5G-Enabled Vehicular Networks: A Distributed Offloading Framework for Traffic Big Data. *IEEE Trans. Ind. Inform.* **2020**, *16*, 1352–1361. [\[CrossRef\]](#)
3. Mahmud, R.; Toosi, A.N.; Ramamohanarao, K.; Buyya, R. Context-Aware Placement of Industry 4.0 Applications in Fog Computing Environments. *IEEE Trans. Ind. Inform.* **2020**, *16*, 7004–7013. [\[CrossRef\]](#)
4. Wang, S.; Chen, H.; Wang, Y. Collaborative Caching for Energy Optimization in Content-Centric Internet of Things. *IEEE Trans. Comput. Soc. Syst.* **2022**, *9*, 230–238. [\[CrossRef\]](#)
5. Xu, Y.; Zhang, T.; Liu, Y.; Yang, D.; Xiao, L.; Tao, M. UAV-Assisted MEC Networks With Aerial and Ground Cooperation. *IEEE Trans. Wirel. Commun.* **2021**, *20*, 7712–7727. [\[CrossRef\]](#)
6. Cao, H.; Lin, Z.; Yang, L.; Wang, J.; Guizani, M. DT-SFC-6G: Digital Twins Assisted Service Function Chains in Softwarized 6G Networks for Emerging V2X. *IEEE Netw.* **2023**, *37*, 289–296. [\[CrossRef\]](#)
7. Liu, Y.; Lu, H.; Li, X.; Zhang, Y.; Xi, L.; Zhao, D. Dynamic Service Function Chain Orchestration for NFV/MEC-Enabled IoT Networks: A Deep Reinforcement Learning Approach. *IEEE Internet Things J.* **2021**, *8*, 7450–7465. [\[CrossRef\]](#)
8. Asgarian, M.; Jamshidi, K.; Bohlooli, A. An Efficient Approximation Algorithm for Service Function Chaining Placement in Edge-Cloud Computing Industrial Internet of Things. *IEEE Internet Things J.* **2024**, *11*, 12815–12822. [\[CrossRef\]](#)
9. Wu, H.; Chen, J.; Nguyen, T.N.; Tang, H. Lyapunov-Guided Delay-Aware Energy Efficient Offloading in IIoT-MEC Systems. *IEEE Trans. Ind. Inform.* **2023**, *19*, 2117–2128. [\[CrossRef\]](#)
10. Lin, B.; Chen, X.; Chen, X.; Ma, Y.; Xiong, N.N. SGCS: An Intelligent Stackelberg-Game-Based Computation Offloading and Resource Pricing Scheme in Blockchain-Enabled MEC for IIoT. *IEEE Internet Things J.* **2024**, *11*, 26727–26740. [\[CrossRef\]](#)
11. Bebertta, S.; Senapati, D.; Panigrahi, C.R.; Pati, B. Adaptive Performance Modeling Framework for QoS-Aware Offloading in MEC-Based IIoT Systems. *IEEE Internet Things J.* **2022**, *9*, 10162–10171. [\[CrossRef\]](#)
12. Sun, L.; Wang, J.; Lin, B. Task Allocation Strategy for MEC-Enabled IIoTs via Bayesian Network Based Evolutionary Computation. *IEEE Trans. Ind. Inform.* **2021**, *17*, 3441–3449. [\[CrossRef\]](#)
13. Chen, Z.; Yu, Z. Intelligent Offloading in Blockchain-Based Mobile Crowdsensing Using Deep Reinforcement Learning. *IEEE Commun. Mag.* **2023**, *61*, 118–123. [\[CrossRef\]](#)
14. Chen, Z.; Xiong, B.; Chen, X.; Min, G.; Li, J. Joint Computation Offloading and Resource Allocation in Multi-Edge Smart Communities With Personalized Federated Deep Reinforcement Learning. *IEEE Trans. Mob. Comput.* **2024**, *23*, 11604–11619. [\[CrossRef\]](#)
15. Chen, Z.; Zhang, J.; Huang, Z.; Wang, P.; Yu, Z.; Miao, W. Computation offloading in blockchain-enabled MCS systems: A scalable deep reinforcement learning approach. *Future Gener. Comput. Syst.* **2024**, *153*, 301–311. [\[CrossRef\]](#)
16. Xu, S.; Li, Y.; Guo, S.; Lei, C.; Liu, D.; Qiu, X. Cloud-Edge Collaborative SFC Mapping for Industrial IoT Using Deep Reinforcement Learning. *IEEE Trans. Ind. Inform.* **2022**, *18*, 4158–4168. [\[CrossRef\]](#)
17. Song, S.; Lee, C.; Cho, H.; Lim, G.; Chung, J.M. Clustered Virtualized Network Functions Resource Allocation based on Context-Aware Grouping in 5G Edge Networks. *IEEE Trans. Mob. Comput.* **2020**, *19*, 1072–1083. [\[CrossRef\]](#)
18. Sun, G.; Xu, Z.; Yu, H.; Chang, V. Dynamic Network Function Provisioning to Enable Network in Box for Industrial Applications. *IEEE Trans. Ind. Inform.* **2021**, *17*, 7155–7164. [\[CrossRef\]](#)
19. Agarwal, S.; Chintapalli, V.R.; Tamma, B.R. FlexSFC: Flexible Resource Allocation and VNF Parallelism for Improved SFC Placement. In Proceedings of the 2022 IEEE 8th International Conference on Network Softwarization (NetSoft), Milan, Italy, 27 June–1 July 2022; pp. 302–306. [\[CrossRef\]](#)
20. Han, Y.; Meng, W.; Fan, W. SFC Placement and Dynamic Resource Allocation Based on VNF Performance-Resource Function and Service Requirement in Cloud-Edge Environment. *J. Syst. Eng. Electron.* **2024**, *35*, 906–921. [\[CrossRef\]](#)
21. Guo, S.; Dai, Y.; Xu, S.; Qiu, X.; Qi, F. Trusted Cloud-Edge Network Resource Management: DRL-Driven Service Function Chain Orchestration for IoT. *IEEE Internet Things J.* **2020**, *7*, 6010–6022. [\[CrossRef\]](#)
22. Quang, P.T.A.; Hadjadj-Aoul, Y.; Outtagarts, A. A Deep Reinforcement Learning Approach for VNF Forwarding Graph Embedding. *IEEE Trans. Netw. Serv. Manag.* **2019**, *16*, 1318–1331. [\[CrossRef\]](#)

23. Alsenwi, M.; Tran, N.H.; Bennis, M.; Pandey, S.R.; Bairagi, A.K.; Hong, C.S. Intelligent Resource Slicing for eMBB and URLLC Coexistence in 5G and Beyond: A Deep Reinforcement Learning Based Approach. *IEEE Trans. Wirel. Commun.* **2021**, *20*, 4585–4600. [[CrossRef](#)]
24. Abedin, S.F.; Munir, M.S.; Tran, N.H.; Han, Z.; Hong, C.S. Data Freshness and Energy-Efficient UAV Navigation Optimization: A Deep Reinforcement Learning Approach. *IEEE Trans. Intell. Transp. Syst.* **2021**, *22*, 5994–6006. [[CrossRef](#)]
25. Chen, H.; Wang, S.; Li, G.; Nie, L.; Wang, X.; Ning, Z. Distributed Orchestration of Service Function Chains for Edge Intelligence in the Industrial Internet of Things. *IEEE Trans. Ind. Inform.* **2022**, *18*, 6244–6254. [[CrossRef](#)]
26. Li, J.; Wang, R.; Wang, K. Service Function Chaining in Industrial Internet of Things with Edge Intelligence: A Natural Actor-Critic Approach. *IEEE Trans. Ind. Inform.* **2023**, *19*, 491–502. [[CrossRef](#)]
27. Pourghasemian, M.; Abedi, M.R.; Hosseini, S.S.; Mokari, N.; Javan, M.R.; Jorswieck, E.A. AI-Based Mobility-Aware Energy Efficient Resource Allocation and Trajectory Design for NFV Enabled Aerial Networks. *IEEE Trans. Green Commun. Netw.* **2023**, *7*, 281–297. [[CrossRef](#)]
28. Luong, N.C.; Hoang, D.T.; Gong, S.; Niyato, D.; Wang, P.; Liang, Y.; Kim, D.I. Applications of Deep Reinforcement Learning in Communications and Networking: A Survey. *IEEE Commun. Surv. Tutor.* **2019**, *21*, 3133–3174. [[CrossRef](#)]
29. Fu, X.; Yu, F.R.; Wang, J.; Qi, Q.; Liao, J. Service Function Chain Embedding for NFV-Enabled IoT Based on Deep Reinforcement Learning. *IEEE Commun. Mag.* **2019**, *57*, 102–108. [[CrossRef](#)]
30. Fu, X.; Yu, F.R.; Wang, J.; Qi, Q.; Liao, J. Dynamic Service Function Chain Embedding for NFV-Enabled IoT: A Deep Reinforcement Learning Approach. *IEEE Trans. Wirel. Commun.* **2020**, *19*, 507–519. [[CrossRef](#)]
31. Huang, H.; Zeng, C.; Zhao, Y.; Min, G.; Zhu, Y.; Miao, W.; Hu, J. Scalable Orchestration of Service Function Chains in NFV-Enabled Networks: A Federated Reinforcement Learning Approach. *IEEE J. Sel. Areas Commun.* **2021**, *39*, 2558–2571. [[CrossRef](#)]
32. Liu, W.; Li, B.; Xie, W.; Dai, Y.; Fei, Z. Energy Efficient Computation Offloading in Aerial Edge Networks With Multi-Agent Cooperation. *IEEE Trans. Wirel. Commun.* **2023**, *22*, 5725–5739. [[CrossRef](#)]
33. Song, F.; Deng, M.; Xing, H.; Liu, Y.; Ye, F.; Xiao, Z. Energy-Efficient Trajectory Optimization With Wireless Charging in UAV-Assisted MEC Based on Multi-Objective Reinforcement Learning. *IEEE Trans. Mob. Comput.* **2024**, *23*, 10867–10884. [[CrossRef](#)]
34. Chen, G.; Zhang, X.; Qi, S.; Zeng, Q.; Zhang, Y.D. Network Slicing Resource Allocation Optimization Based on Multiactor-Attention-Critic Joint With Bidding in Heterogeneous Integrated Network. *IEEE Syst. J.* **2024**, *18*, 1186–1197. [[CrossRef](#)]
35. Shahab, M.H.; Sharma, Y.; Jindal, A.; Al-Dulaimy, A. A Bi-Objective Policy for Resilient and Sustainable SFC Management in Telco-Cloud Environments. *IEEE Access* **2025**, *13*, 215453–215473. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.